



Karnataka State Police
Government of Karnataka

Powered by **I12S**

Dataathon 2026

Technology partner



Team Details

- a. **Team name:** Rangdari Gang
- b. **Team leader name:** Soma Mishra
- c. **Team size:** 2 (Sushanth Yelishetty and Soma Mishra)
- d. **Problem Statement:** AI-Driven Crime Analytics & Visualization Platform

Brief about the solution

Drishti links undetected cases by how the crime was done, not by who was named.

Karnataka carries thousands of undetected cases (final report type C). They sit in isolation because the FIR schema has **no global offender identity** — Accused.PersonID is documented as “A1, A2, A3...”, a sort order within a single FIR. When one offender works across several stations the database holds several unrelated name strings, and nobody joins them.

For every undetected FIR the engine builds a **behavioural fingerprint** from seven weighted signals — modus operandi text, offence type, the Act and Section invoked, location, time-of-day pattern and victim profile. It scores every pair of cases, then groups the look-alikes using **agglomerative clustering** on the resulting distance matrix. Each pair's score is discounted by how much evidence was actually available, so thin data makes the engine more cautious rather than louder.

The output is a **ranked queue of suspected offender groups**, each citing real FIR numbers and showing which signals produced the link. From **789 undetected cases across 69 stations and 19 districts** it surfaces **16 groups holding 87 cases** — **every one spanning more than one police station** — then projects where each may strike next.

Opportunities

a. How different is it from any of the other existing ideas?

Typical crime tools

→ Drishti

- Descriptive dashboards showing where crime happened → **Prescriptive — surfaces hidden links and actionable case series to reopen**
- Depend on offender IDs or manual case linking → **Works without any person ID, using behavioural fingerprinting**
- Black-box predictions with limited transparency → **Explainable evidence trail, measured live — P 0.928 / R 0.965 / F1 0.946**
- Perform poorly on sparse free-text reports → **Degrades safely — weights renormalise and scores are discounted by evidence coverage**
- Rely on external LLM APIs, with per-query cost and off-premises processing → **On-premises friendly — deterministic scikit-learn, no LLM API, data stays in Catalyst**

b. How will it be able to solve the problem?

An investigating officer sees their own station's cases. Nobody is positioned to notice that four robberies in four different stations share a method, an hour and a radius. Drishti runs that comparison statewide, with the evidence attached — then closes the loop forward, scoring each newly registered FIR against every known group as it lands, so a case joins a pattern **before** it is worked as an isolated file.

c. USP of the proposed solution

Works without an offender ID — linkage is behavioural, so it runs on the existing FIR schema unchanged.

Every claim is evidence-backed — per-signal contributions and cited CrimeNo values. If the engine cannot cite it, it does not say it.

It does not invent patterns. Scramble the data so nothing is there and it returns **zero groups, on every run** — shown live in the product.

Identity carries zero weight, and we proved it should — letting the accused name influence linkage made the engine measurably worse.

Detection becomes prevention — a projected window and area for each group's next offence, back-tested at 92.9% on area, written out in plain language.

List of features offered by the solution

Undetected case linkage — behavioural fingerprint over seven weighted signals, composite pairwise similarity discounted by evidence coverage, agglomerative clustering into ranked offender groups

Evidence trail on every link — per-signal contribution, cited FIR numbers, spatial span and a full case timeline

Forecast — its own workspace — every group's projected window, area radius and usual hours, ranked with open windows first and written out as a plain-language report

Live FIR triage — an incoming case scored against every known group, graded against that group's own cohesion — Escalate, Review, or treat as unlinked

Association network — 25 people who never share an FIR, linked by shared modus operandi in overlapping jurisdictions; click one and their whole crime history draws on the map in date order

Interactive 3D district map — districts extruded from real boundaries, with case pins, link webs, forecast rings and red/yellow risk zones

Robustness shown in-product — negative control and graceful-degradation results, recomputed live alongside accuracy

Printable case brief — server-rendered PDF dossier with evidence, timeline, forecast and signature block

Role-based access control — investigator, analyst and supervisor, enforced at the API gateway; investigators scoped to their own district

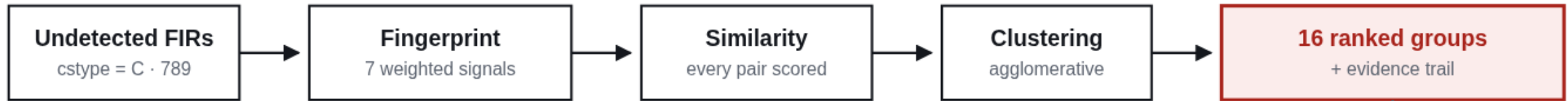
Natural-language search — retrieval-only, so every answer cites real records and cannot fabricate

Bilingual interface — English and Kannada across all labels, explanations and the written forecast reports

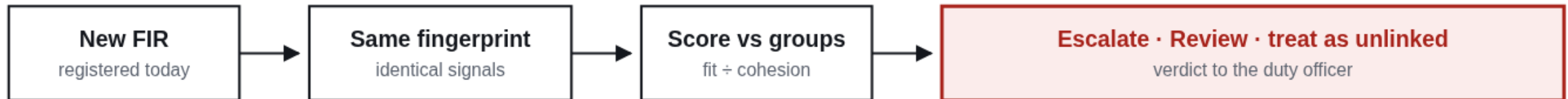


Process flow diagram

ARCHIVE PASS · RECOMPUTED NIGHTLY



LIVE INTAKE · PER NEW FIR



The same fingerprint runs in both passes — that is what lets a brand-new FIR be judged against groups built from the archive, and why the two lanes are one system rather than two.

Archive pass — every undetected FIR is fingerprinted on seven weighted signals, scored against every other case, and clustered. Groups are ranked by cohesion multiplied by actionability, favouring jurisdictional reach, recency and gravity. Runs nightly.

Live intake — a newly registered FIR is fingerprinted identically and scored against each group, graded against that group's own internal cohesion rather than a fixed threshold.

Nothing is asserted. Each group carries the per-signal contributions that produced it and the real FIR numbers it cites, so an investigator can audit the reasoning.



Wireframes / Mock diagrams of the proposed solution

Command bar

retrieval-only search;
every answer cites a real FIR

Primary navigation

nine analytical views;
locked by role

Forecast — its own tab

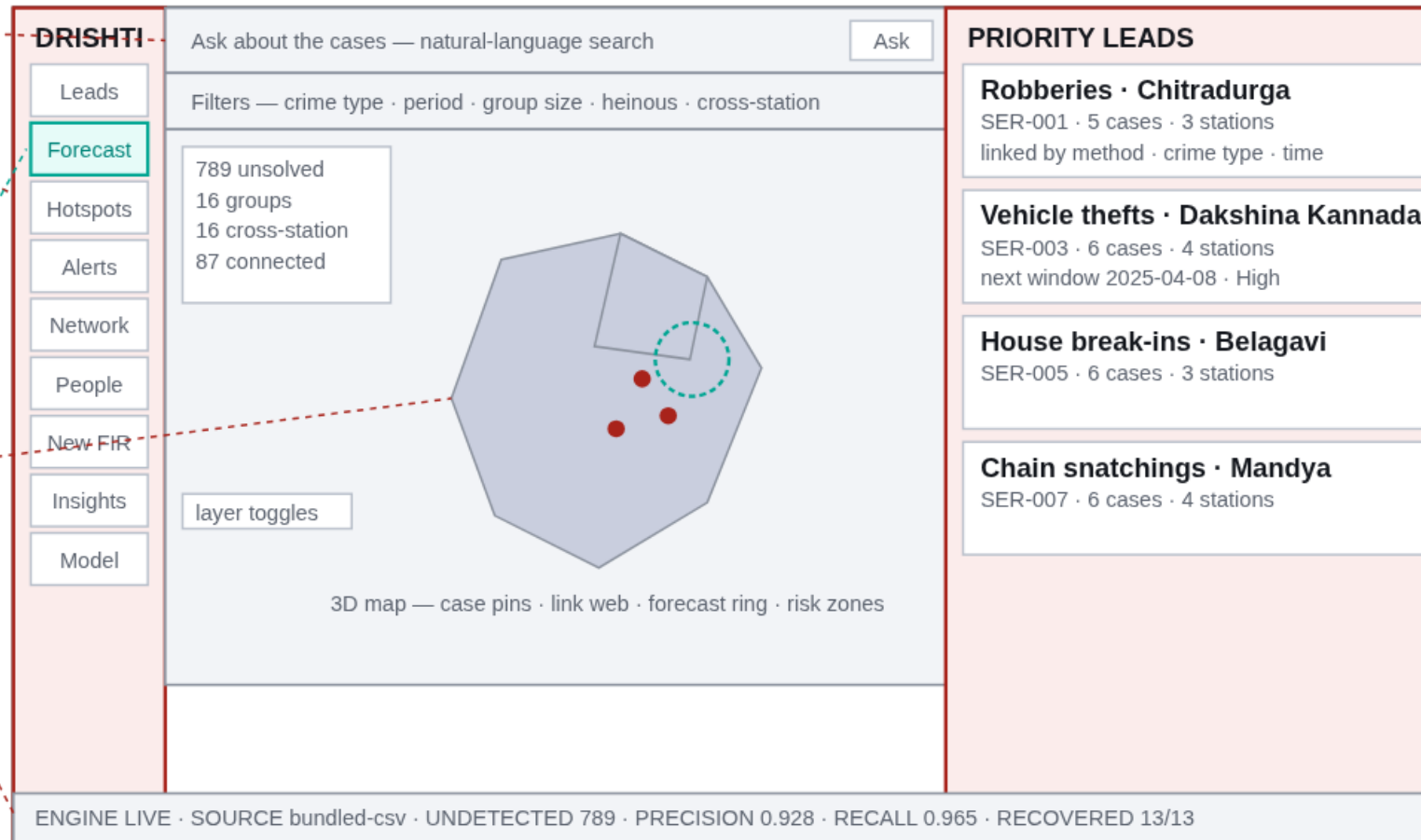
every group's projected
window and area

Map stage

districts extruded by case
volume; teal ring is the
projected next-strike zone

Status bar

live engine state and
measured accuracy



Dossier panel

ranked leads; opens
to a full case file

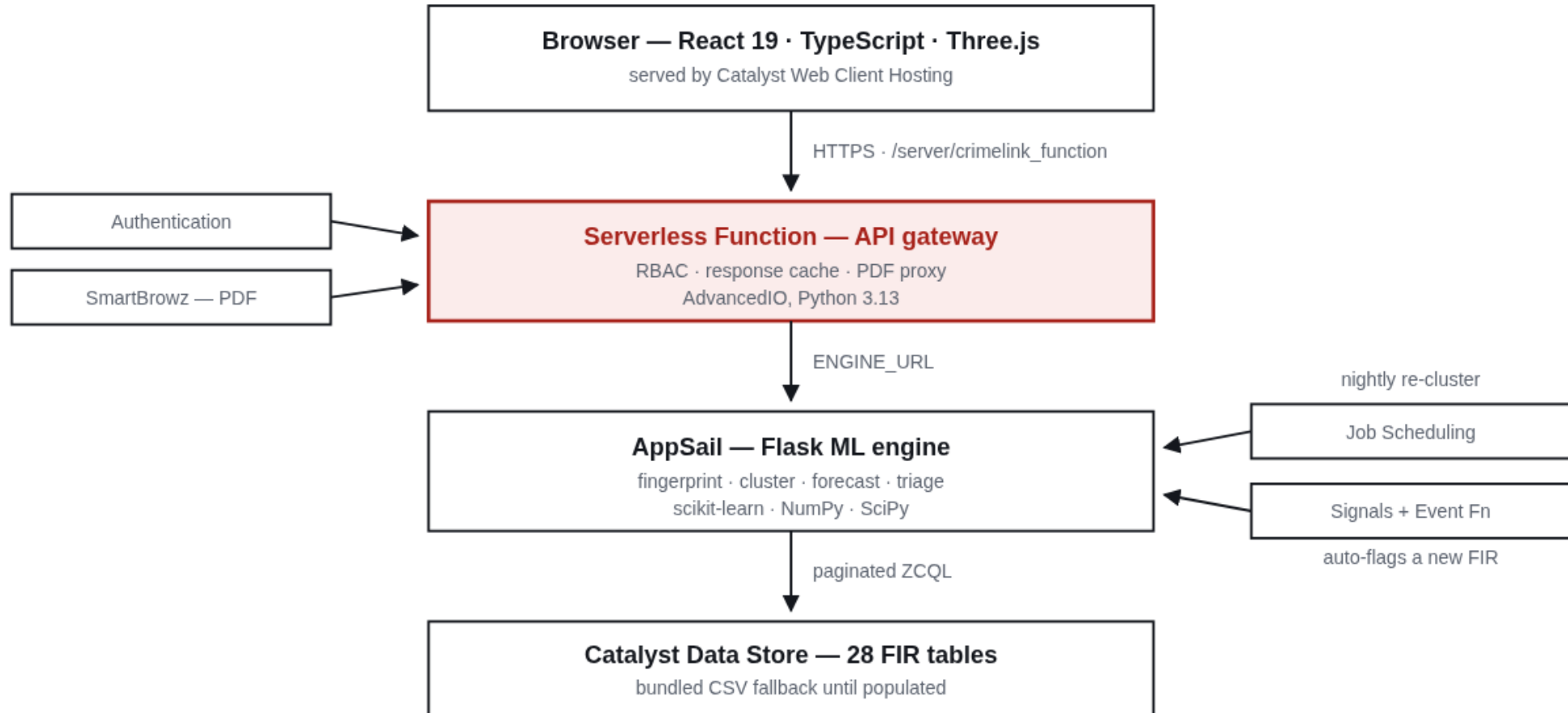
Evidence on every card

why it linked, and
the projected window

Three regions, each doing one job: navigation on the rail, the map as the workspace, and the dossier where a lead opens into a full case file with its evidence. Live screens of this layout follow.



Architecture diagram of the proposed solution



Access control sits at the gateway, so no request reaches case data before a role is resolved. The engine is the only component that touches the Data Store.

Technologies to be used in the solution

Frontend	React 19, TypeScript, Three.js with React-Three-Fiber (WebGL 3D map)
Machine learning	Python 3.13, scikit-learn (TF-IDF, AgglomerativeClustering), NumPy, SciPy, NetworkX
API layer	Flask on AppSail; Catalyst Serverless Function (AdvancedIO) as the gateway
Document generation	fpdf2 for the server-rendered case brief; SmartBrowz as the Catalyst-native renderer
Data	Catalyst Data Store queried with ZCQL; schema-faithful CSV fallback
Geospatial	Karnataka district boundary GeoJSON, Douglas-Peucker simplification, haversine distance

Techniques

Behavioural fingerprinting · TF-IDF over complaint text · weighted composite similarity with per-pair renormalisation and an evidence-coverage discount · agglomerative clustering on a precomputed distance matrix · leave-one-signal-out ablation · hold-one-out forecast back-testing · permutation-based negative control · Douglas-Peucker simplification for the district meshes.

Design constraints we held to

- Clustering stays on **scikit-learn primitives**, so the engine installs from requirements.txt on the Catalyst managed runtime with **no native compilation**.

No external LLM API. Retrieval is deterministic, so no case data leaves the Catalyst environment and there is no per-query cost.

CPU only — no GPU anywhere in the pipeline.

Catalyst Services being used in the solution

AppSail	Hosts the ML engine — fingerprinting, clustering, forecast, triage, robustness, PDF
Serverless Functions	API gateway; enforces role-based access before any case data is read
Data Store	The FIR schema — 28 tables, queried by ZCQL
Authentication	Identity behind the investigator / analyst / supervisor roles
Cache	Caches computed API responses across function instances
Job Scheduling	Nightly re-cluster and cache flush — used instead of Cron, which is deprecated
Signals + Event Functions	Auto-triages a newly registered FIR — instead of Event Listeners, also deprecated
SmartBrowz	Renders the case-brief PDF
Web Client Hosting	Serves the React application

We deliberately avoided the deprecated services: **Job Scheduling replaces Cron** and **Signals with Event Functions replaces Event Listeners**, both end-of-life 30 April 2026.

Since the previous submission, four services moved from roadmap to shipped — Authentication (role-based access control), Job Scheduling, Signals with Event Functions, and SmartBrowz. Data Store full-text search on BriefFacts activates when the tables are populated.

Estimated implementation cost

No new infrastructure and no new data collection. Costs scale gradually with usage.

Pilot / demo stage — runs on the Catalyst Developer Tier at roughly ₹0, within the free tier.

District-level deployment ($\sim 10^4$ – 10^5 FIRs) — AppSail, Serverless Functions and Data Store, in the **low thousands of rupees per month**.

State-wide ($\sim 10^6$ FIRs with nightly recomputation) — **tens of thousands of rupees per month**, driven mainly by AppSail and Data Store scaling.

Costs stay low because the pipeline uses **CPU-based scikit-learn models** — **no GPUs and no external LLM API calls** — and linkage runs as a nightly batch rather than on every user query. Figures are estimates and vary with data volume and recomputation frequency.

What is not in the cost

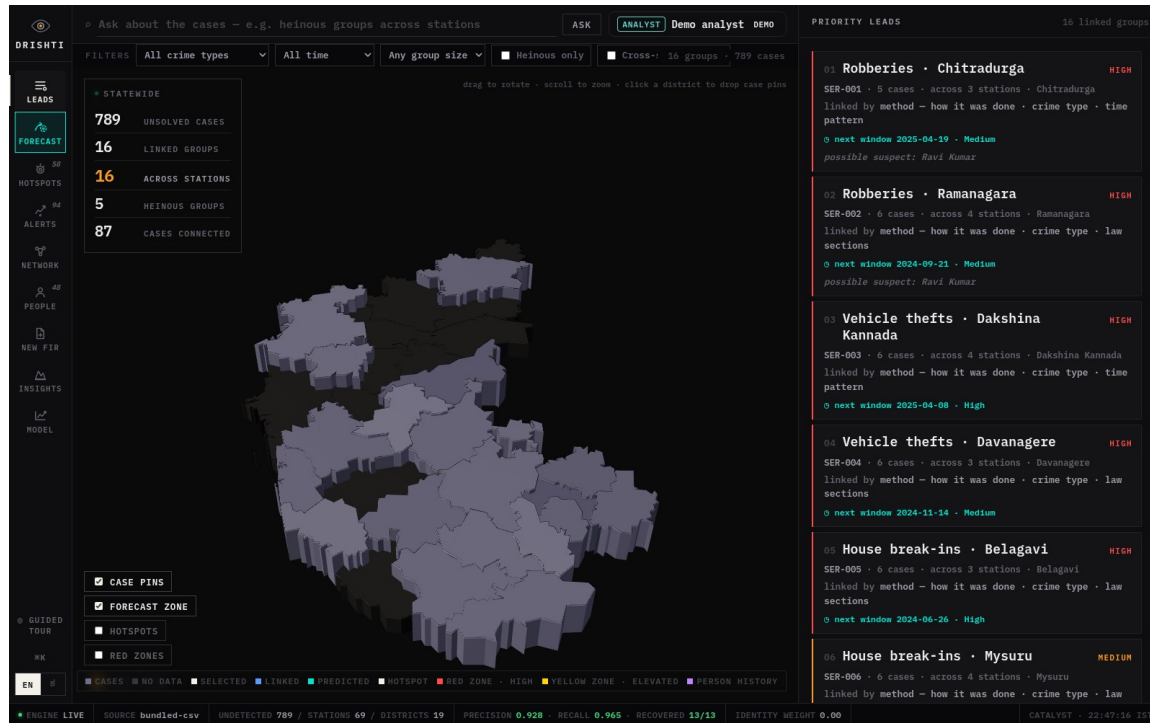
No new data collection — it reads the FIR schema exactly as it exists.

No change to officer workflow — no additional field, form or step when filing an FIR.

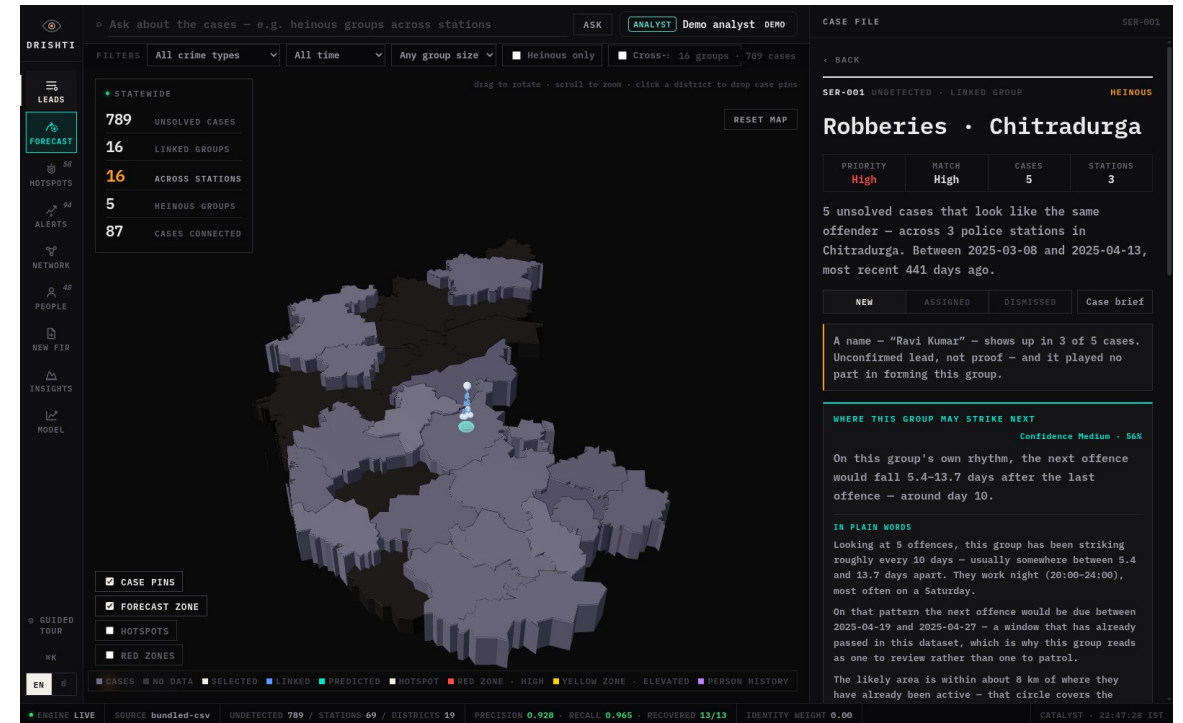
No third-party licences — every component is open-source or a Catalyst service already in the KSP stack.

No GPU or inference hardware, and no per-query API spend that grows with usage.

Snapshots of the prototype

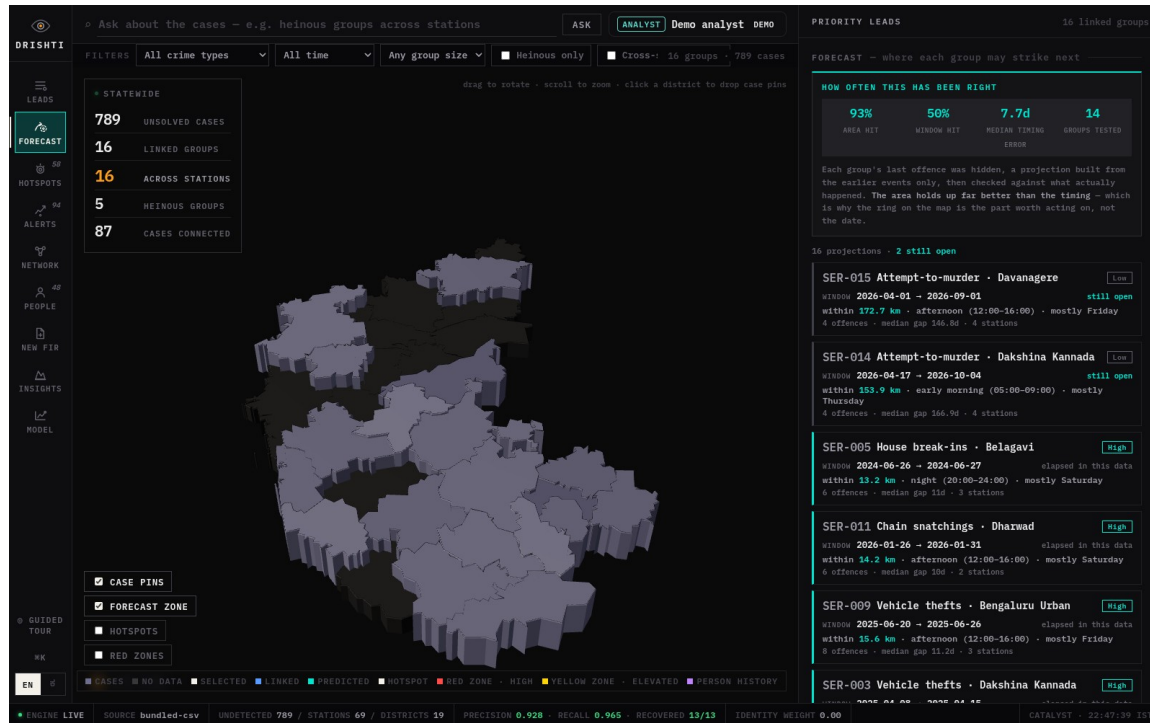


Statewide — 789 undetected cases, 16 linked groups, all cross-station

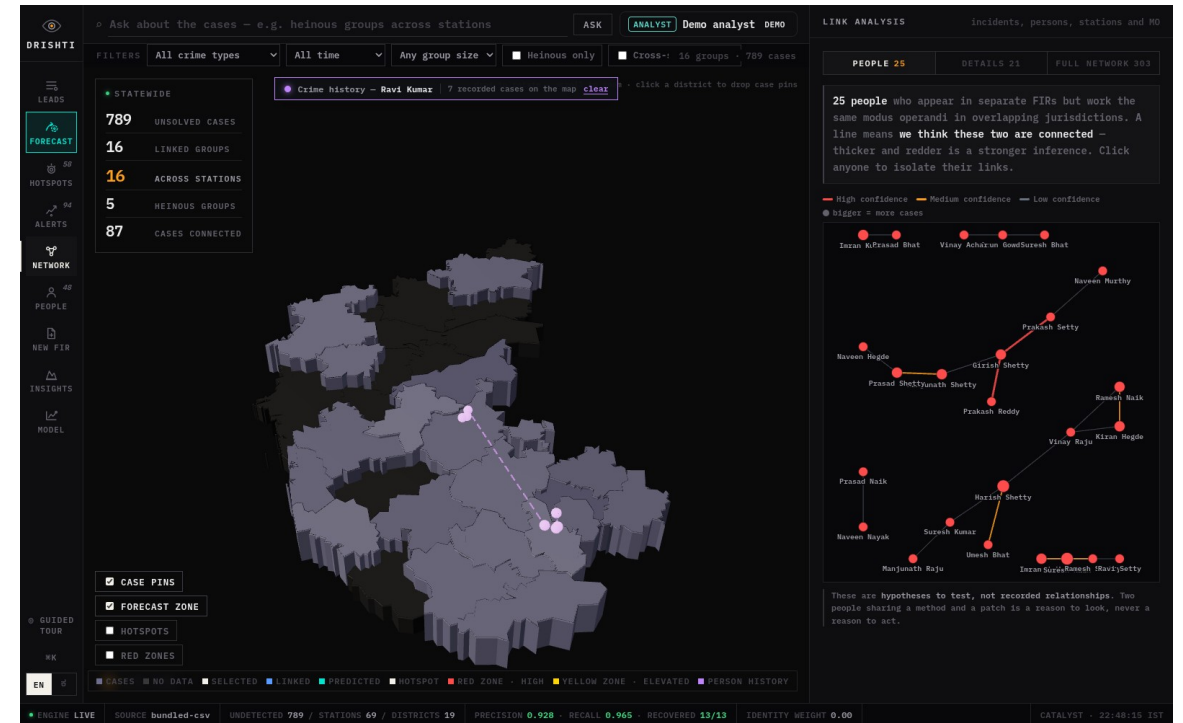


A linked group — evidence trail and the projection written in plain words

Snapshots of the prototype

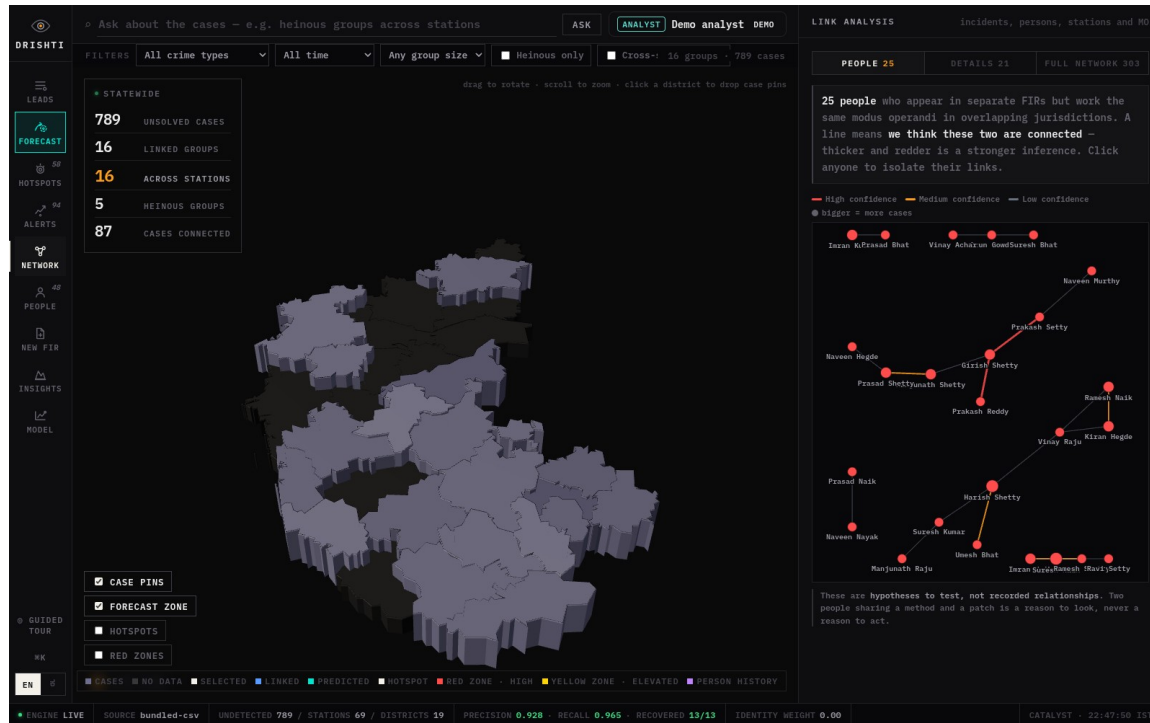


Forecast — all 16 projections ranked, open windows first

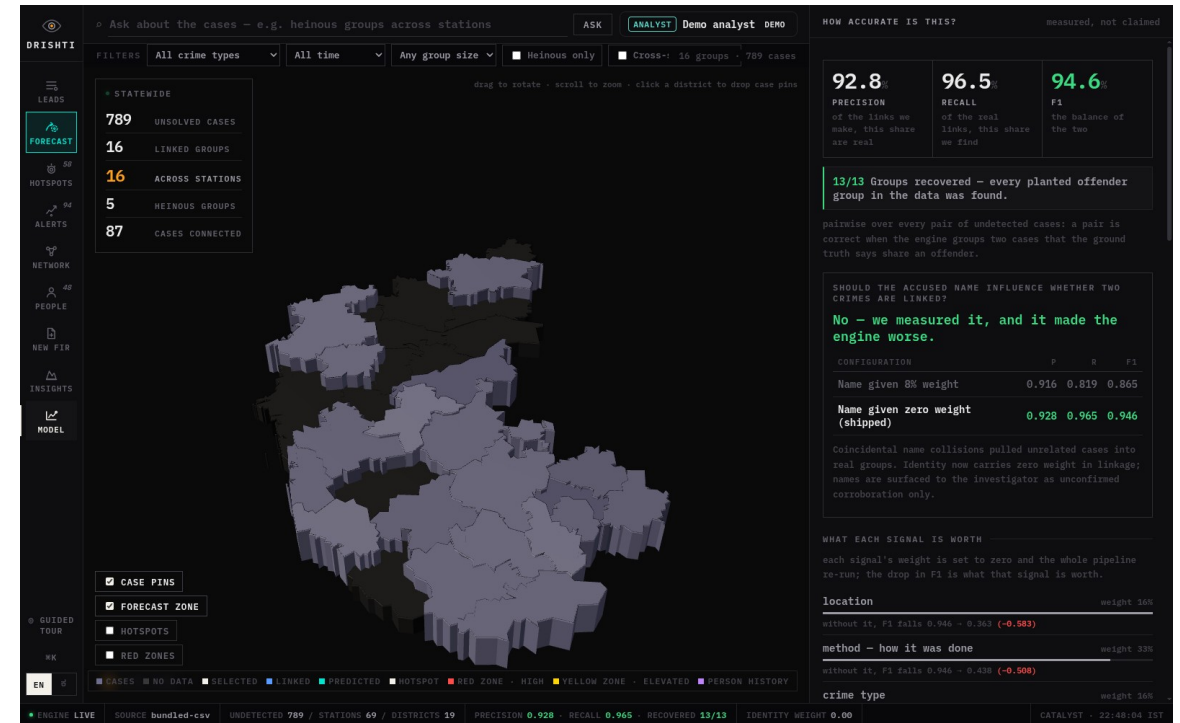


One person's whole crime history drawn on the map in date order

Snapshots of the prototype



Association network — 25 people linked by shared modus operandi



Measured accuracy, ablation, and the robustness checks



Prototype Performance report / Benchmarking

Linkage accuracy — pairwise over every pair of undetected cases against planted ground truth

Precision 0.928 · Recall 0.965 · F1 0.946 · 13 of 13 planted groups recovered. Reproducible offline in one command: `python appsail-python/run_local.py`

Does it invent groups? — the negative control

Every behavioural field is shuffled independently, so all the same texts, times, places and crime types remain in the same proportions and only the link between a case and its own behaviour is destroyed. No offender pattern can survive.

16 groups on the real data. 0, 0 and 0 across three scrambled runs. The engine reports nothing when there is nothing to find — which is what makes the accuracy figures above worth reading.

What happens when the free-text description is missing?

Full text → **16 groups, P 0.928, 13/13 recovered.** 50% blank → 15 groups, P 0.778, 10/13. 100% blank → 14 groups, P 0.630, 7/13.

Fewer groups with lower recall is the intended behaviour: on thin evidence the engine should get more cautious, not louder. Without the evidence-coverage discount this returned **53 groups at precision 0.375** — a flood of false leads.

Performance report / Benchmarking (continued)

What each signal is worth — leave-one-out ablation

Each signal's weight is set to zero and the whole pipeline re-run; the drop in F1 is what that signal is worth.

Location removed → F1 falls 0.946 to **0.363**. Method, the MO text → **0.438**. Crime type → **0.737**. Who was targeted → **0.780**.

The identity experiment

Accused name at 8% weight → P 0.916 / R 0.819 / F1 0.865. **At zero weight, as shipped → P 0.928 / R 0.965 / F1 0.946.**

Coincidental name collisions pulled unrelated cases into real groups. Identity now carries no weight in linkage at all; a recurring name is shown to the investigator as unconfirmed corroboration only.

Forecast, back-tested

Each group's last offence was held out and a projection built from the earlier events only, then checked against what actually happened: **area hit rate 92.9%**, window hit rate 50.0%, median timing error 7.7 days, across 14 groups. The area holds up far better than the timing — which is why the ring on the map is the actionable part of the projection, not the date.

Schema conformance and the data these figures come from

28 of 28 ER tables implemented, all foreign keys resolve. The KSP dataset was not released, so these figures are measured on schema-faithful synthetic data with planted offender groups. Real FIR data swaps in through the same tables and pipeline unchanged, and the accuracy panel re-measures itself against it.



Provide links to your:

1. GitHub Public Repository

<https://github.com/whysush/drishti-crimelink>

2. Demo Video Link (3 minutes)

https://drive.google.com/file/d/1EJFTYKawu_4t-UEj-A0NRav7H7KGvQNz/view?usp=sharing

3. Deployed Link

<https://crimelink-927199644.development.catalystserverless.com/app/index.html>

What is in each

Repository — the full linkage engine, forecast, triage, person resolution and robustness checks; the API gateway with its access-control layer; the job and event functions; the React client; and the synthetic data generator with its ground truth, so every figure on the previous slide can be reproduced.

Deployed link — open it a minute or two before viewing. The engine idles between uses and the status bar reads ENGINE LIVE once warm.

Deep links worth trying — ?tab=model (accuracy and robustness), ?tab=forecast (all projections), ?tab=network (the association graph), ?role=investigator&district=577 (access control), ?lang=kn (Kannada).

Additional Details / Future Development

Go live on real FIR data. The Data Store schema and loader are built and the engine already has a Data Store backend; populating the 28 tables switches the source with no code change, and enables full-text search on BriefFacts.

Promote to a Production environment. The prototype runs on Catalyst Development.

Close the loop on the auto-flag. The Signals event function is deployed; binding it to the FIR insert event turns triage from a screen an officer opens into something that happens the moment a case is registered.

Kannada voice. The interface and the written forecast reports are already bilingual; Zia Services would add speech input and translation of free-text complaints.

Learn from investigator feedback. Every group can be marked assigned or dismissed — those judgements are the labels needed to tune signal weights against real outcomes rather than planted ground truth.

Where the method fails — stated on the slide deliberately

Links are **inferred, never asserted** — a lead to investigate, not evidence. Two unrelated offenders working the same market at the same hour can look alike. Only 233 of 789 cases name an accused at all, which is why name recurrence is a lead rather than a conclusion. Complainant caste and religion exist in the schema and are **never loaded into the engine at all**.



Karnataka State Police
Government of Karnataka

Powered by **1125**
HACKSRELL

Datathon

2026

Technology partner



THANK YOU